

BACHELOR PAPER

Thesis submitted in fulfillment of the requirements for the degree of Bachelor of Science in Engineering at the University of Applied Sciences Technikum Wien - Degree Program BIF

Development and Evaluation of a Multimodal AI-Assisted Reading Platform for Second Language Acquisition

By: Mikail Kaya

Student Number: 2310257196

Supervisor: Mag. rer. soc. oec. Edin Pezerovic, BSc
(WU) MSc (WU) MSc

Wien, May 18, 2026

Declaration

“As author and creator of this work to hand, I confirm with my signature knowledge of the relevant copyright regulations governed by higher education acts (see Urheberrechtsgesetz /Austrian copyright law as amended as well as the Statute on Studies Act Provisions / Examination Regulations of the UAS Technikum Wien as amended).

I hereby declare that I completed the present work independently and that any ideas, whether written by others or by myself, have been fully sourced and referenced. I am aware of any consequences I may face on the part of the degree program director if there should be evidence of missing autonomy and independence or evidence of any intent to fraudulently achieve a pass mark for this work (see Statute on Studies Act Provisions / Examination Regulations of the UAS Technikum Wien as amended).

I further declare that up to this date I have not published the work to hand nor have I presented it to another examination board in the same or similar form. I affirm that the version submitted matches the version in the upload tool.“

Wien, May 18, 2026

Digital Signature

Kurzfassung

Diese Bachelorarbeit präsentiert das Design, die Implementierung und die Evaluation einer multimodalen, KI-gestützten Leseplattform, die Lernende einer Fremdsprache beim Verständnis authentischer Texte unterstützt. Das System ermöglicht das Hochladen und Verwalten von Dokumenten, die verarbeitet und in einer interaktiven Leseoberfläche angezeigt werden. Innerhalb dieser Oberfläche können Nutzer einzelne Wörter auswählen und kontextbezogene Unterstützung in Form von Erklärungen, Übersetzungen, Sprachausgabe und KI-generierten Bildern erhalten.

Die Anwendung ist als webbasiertes System mit einer mehrschichtigen Architektur realisiert, bestehend aus einem Spring-Boot-Backend, einem JavaScript-basierten Frontend, einer PostgreSQL-Datenbank und einem MinIO-Objektspeicher. Externe KI-Dienste sind über APIs integriert und werden vom Backend orchestriert, um multimodales Feedback in Echtzeit bereitzustellen.

Die Entwicklung folgt einem Design-Science-Research-Ansatz mit Fokus auf iterativem Systemdesign und iterativer Implementierung anstelle von Hypothesentests. Die Evaluation erfolgt durch funktionale Verifizierung, Anforderungsrückverfolgbarkeit und eine heuristische Usability-Analyse nach den Prinzipien von Nielsen. Die Ergebnisse zeigen, dass die Integration mehrerer KI-Dienste in eine einheitliche Leseumgebung technisch machbar ist und einen nutzbaren Prototyp für interaktive Sprachunterstützung bietet.

Das System ist zwar nicht als vollständige Sprachlernplattform konzipiert, zeigt aber Potenzial als unterstützendes Werkzeug für den Vokabelerwerb und das Leseverständnis. Zu den Einschränkungen zählen die Abhängigkeit von externen KI-Diensten und die eingeschränkte Unterstützung komplexer Dokumentstrukturen. Diese Einschränkungen gelten für die aktuelle Prototyp-Implementierung und könnten in zukünftigen Versionen durch die Integration lokal gehosteter Open-Source-Modelle anstelle externer APIs gemildert werden. Dadurch würden Unabhängigkeit, Skalierbarkeit und Datenschutz verbessert. Zukünftige Arbeiten könnten zudem die Dokumentenverarbeitung und die Systemflexibilität weiter optimieren.

Schlagworte: Multimodale KI, Leseassistent, Sprachenlernen, Webanwendung

Abstract

This bachelor thesis presents the design, implementation, and evaluation of a multimodal reading platform assisted by Artificial Intelligence (AI) to support second-language learners in understanding authentic texts. The system enables users to upload and manage documents, which are processed and displayed in an interactive reading interface. Within this interface, users can select individual words to receive contextual assistance, including explanations, translations, text-to-speech output, and AI-generated images.

The application is architected as a web-based system using a layered architecture consisting of a Spring Boot backend, a JavaScript-based frontend, a PostgreSQL database, and MinIO object storage. External AI services are integrated via APIs and orchestrated by the backend to provide multimodal feedback in real time.

The development follows a Design Science Research approach, focusing on iterative system design and implementation rather than hypothesis-testing. Evaluation is conducted through functional verification, requirements traceability, and a heuristic usability assessment based on Nielsen's 10 usability heuristics. The results demonstrate that integrating multiple AI services into a unified reading environment is technically feasible and provides a usable prototype for interactive language support.

While the system is not intended as a full language-learning platform, it shows potential as a supportive tool for vocabulary acquisition and reading comprehension. Limitations include reliance on external AI services and restricted document structure support. These constraints could be mitigated in future versions by integrating locally hosted open-source models to improve independence, scalability, and privacy. Future work may further enhance document processing and system flexibility.

Keywords: multimodal AI, reading assistant, language learning, web application

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Tasks	2
1.3	Research Question	2
1.4	Background and State of the Art	2
2	Methodology	4
2.1	Research Approach	4
2.2	Problem Identification	4
2.3	Requirements Engineering Approach	5
2.4	System Design and Development Approach	5
2.5	Evaluation Methods	6
3	Solution	7
3.1	Requirements and Analysis Results	7
3.1.1	Stakeholder Analysis	7
3.1.2	User Stories	7
3.1.3	Use Cases	7
3.1.4	Functional and Non-Functional Requirements	8
3.2	System Overview and Functionality	10
3.3	User Interface	10
3.4	Software Architecture	12
3.5	Implementation	14
3.5.1	Technology Stack	14
3.5.2	Backend Implementation	14
3.5.3	Frontend Implementation	15
3.5.4	Document Processing Pipeline	16
3.5.5	AI Service Orchestration	17
3.5.6	Deployment and Development Environment	18
3.6	Evaluation Results	19
3.6.1	Requirements Traceability Matrix	19
3.6.2	Heuristic Evaluation Results	21
4	Discussion	23
4.1	Critical Reflection and Limitations	23

4.2	Fulfilment of Research Goals	24
4.3	Generalisation and Transferability	24
4.4	Retrospective and Lessons Learned	25
	Bibliography	26
	List of Figures	32
	List of Tables	33
	Documentation table of AI-based tools	34
	List of Abbreviations	35
A	Project Setup and Execution	36
A.1	Running the Application (Docker)	36
A.2	Modifying the Project	37
B	Data Model and Persistence	39
B.1	Entity Overview	39
B.2	Relationships and Schema	39
C	User Interface Design Artefacts	41
D	AI Prompt Design	43
D.1	Translation and Explanation Prompt	43
D.2	Image Generation Prompt	43
D.3	Text-To-Speech Prompt	43
E	Heuristic Evaluation Details	45

1 Introduction

The following section outlines the motivation and objectives of the work, together with the underlying research question and context.

1.1 Motivation

Broad reading is widely recognised as an essential driver of second-language (L2) acquisition, significantly improving reading comprehension among various learner backgrounds and contexts [1]. In addition to supporting overall understanding, reading plays a central role in lexical development, as repeated exposure to terms in varying contexts helps students recognise definitions more efficiently and interpret language more fluently [2]. These effects tend to be amplified in online settings, in which tools such as integrated dictionaries and flexible systems further enhance vocabulary attainment and lead to lasting educational gains [3]. Beyond lexical knowledge, literacy activities further contribute to improvements in orthography and syntactic understanding, indicating that it supports a higher comprehensive development of linguistic skills [4]. However, successful understanding is strongly dependent on lexical proficiency, as research indicates a direct correlation between word knowledge and comprehension, with students typically needing to know around 98% of the terms in a passage to achieve a high degree of clarity, especially in complex materials [5].

Furthermore, text understanding is not determined by lexical expertise alone, but is likewise strongly influenced by cognitive function. When a passage contains too many unfamiliar terms, the resulting cognitive strain may overwhelm active recall, making it more difficult to handle data, retrieve known vocabulary, and connect grammatical frameworks, ultimately impairing reading performance [6]. Educational approaches that aim to lessen this mental burden—such as lowering split attention or limiting inefficient dictionary utilisation—can therefore enhance reading fluency and general language acquisition [6].

Emotional variables, particularly reading-related stress, can significantly hinder L2 learners' understanding and motivation [7]. Challenging texts can trigger tension, further decreasing reading efficiency, but flexible settings supported by Artificial Intelligence (AI) have been shown to lower this burden and encourage more effective reading habits [8]. This highlights the need for tools that provide personalized support to make reading in L2 less stressful and more accessible.

1.2 Tasks

The primary goal of this work is to develop a functioning prototype of a multimodal AI-assisted reading platform, named *Alovre*, that helps users improve comprehension in L2. The system is designed to support learners by providing integrated textual explanations, auditory feedback, and visual scaffolding, thereby making reading more approachable and engaging. By enabling users to work with their own documents and receive on-demand assistance, the platform aims to lower cognitive and emotional barriers to reading, ultimately fostering more effective and self-directed language learning.

1.3 Research Question

Based on the identified challenges in L2 reading and the potential of AI-assisted support, this thesis addresses the following research question:

How can a multimodal AI-assisted reading system be designed and implemented to support second-language learners, and to what extent is the resulting system usable?

To answer this question, the work focuses on the development of a functional prototype and its subsequent evaluation in terms of usability and requirement fulfilment.

1.4 Background and State of the Art

Modern scholarship increasingly highlights the role of AI, specifically Large Language Models (LLMs) such as ChatGPT, in supporting L2 development. A survey of seventy papers shows that such tools mainly offer personalized feedback, customized resources, and structured assistance for language production [9, pp. 1–2]. While much of the focus has been on writing—helping with tasks from idea generation to grammar and error analysis—and oral practice through simulated guidance or augmented environments [9, pp. 12, 17], AI applications for reading are now emerging. Such systems offer real-time lexical explanations, tailored comprehension tasks, and high-quality textual content, while simultaneously enhancing learner drive through adaptive input and on-demand assistance [9, pp. 18–19]. Even with such benefits, doubts remain regarding the reliability of AI-produced content and possible impacts on academic honesty [9, pp. 1–2].

Building on AI-driven explanations, AI increasingly supports non-native readers with various forms of textual and verbal aid. Chatbots powered by LLMs provide immediate definitions, contextual examples, evaluations, and interactive guidance, improving lexical comprehension, usage, and durable memory [10]. Vocal assistance, including text-to-speech (TTS), can enhance understanding of complex passages, particularly for L2 learners with dyslexia [11], and AI-driven literacy programs have shown gains in reading mastery, engagement, and autonomous learning [12]. Beyond auditory support, AI further offers visual scaffolding by trans-

forming learner information to clear dashboards and summaries, enabling individualized feedback and informed instructional choices [13]. In literacy-focused software, programs such as the IRead system embed thematic graphics—including structural maps and AI-generated guiding inquiries—directly into the textual process, boosting interest and aiding deep comprehension while keeping readers focused on the content [14].

To contextualise current technical capabilities, a comparative analysis of leading digital reading assistants was conducted, focusing on the integration of textual, auditory, and visual support. The results indicate that while many platforms provide robust text and audio assistance, none utilise generative AI to offer dynamic, contextual visualisations [15], [16], [17], [18]. Static visual aids, such as "illustrated dictionaries," have mostly been phased out [19], leaving a notable gap in multimodal support. To evaluate these tools, support features are categorised by their use of first-language (L1) glosses (definitions in the learner's native language) and L2 glosses (explanations in the target language). Table 1 summarises the key features of prominent L2 reading tools, illustrating that an integrated platform combining LLM-driven explanations with real-time AI-generated imagery remains unique and necessary.

Table 1: Feature comparison of prominent L2 reading platforms.

Feature	LingQ [15]	Readlang [16]	MS Imm. Reader [17]	LingoReader [18]	Proposed System Alovre
User-Uploaded Content	Yes	Yes	No	No	Yes
L1 Glosses	Yes	Yes	Yes	Yes	Yes
L2 Glosses	Partial ¹	Yes	No	Yes	Yes
Generative Images	No	No	No	No	Yes
TTS	Yes	Yes	Yes	Yes	Yes

¹LingQ typically redirects users to external dictionary tabs for detailed monolingual explanations.

2 Methodology

The methodological foundation of Alovre is presented in the following sections, describing how the platform was systematically developed and evaluated.

2.1 Research Approach

The research approach chosen for this work is Design Science Research (DSR), specifically following the methodology outlined by Peffers et al. [20].

The methodology was selected because it aligns closely with the goals of this thesis: developing a functioning, AI-assisted reading platform that addresses specific challenges in L2 learning. DSR provides a structured process with clear links between problem analysis, solution design, implementation, and evaluation.

Following the DSR process model, the work began with problem identification through a narrative literature review and comparative analysis of existing tools. Based on these findings, solution objectives and system requirements were derived using stakeholder analysis, user stories, use cases, as well as functional and non-functional requirements.

The subsequent design and development phase translated these requirements into a technical solution through iterative implementation, layered architectural design, UML-based system modelling, and interface prototyping. This process resulted in a functional multimodal AI-assisted reading prototype comprising document processing, AI orchestration, and interactive reading-support features. Finally, the resulting artifact was evaluated through functional testing, requirements traceability analysis, and heuristic usability evaluation to assess requirement fulfilment and overall usability.

2.2 Problem Identification

To establish a solid understanding of the research domain, a narrative literature review was conducted, focusing on L2 reading, digital reading platforms, and AI-assisted language learning. This approach supports the identification of key trends, challenges, and research gaps within a field [21], making it suitable for defining the problem space addressed in this thesis.

Relevant publications were analyzed to provide the theoretical foundation for the work and to identify limitations in existing approaches. The findings subsequently informed the definition of system requirements and design decisions for the proposed solution.

In addition, a comparative analysis of existing applications was performed to examine the current state of digital reading assistants. This approach enables software solutions to be systematically evaluated against defined criteria, making it possible to assess their capabilities and identify practical limitations [22].

A selection of representative platforms was analyzed based on a structured set of features, including vocabulary support, TTS functionality, and visual assistance. The findings were consolidated in a comparison table, enabling a clear identification of missing functionalities and highlighting gaps in current systems, particularly in relation to multimodal and AI-driven support.

2.3 Requirements Engineering Approach

To systematically capture and structure the system requirements, an integrated approach combining stakeholder analysis, use cases, user stories, and formal requirement definitions was employed [23, pp. 79–80, 115, 125–126]. This phase began with a lightweight stakeholder analysis to identify primary participants and determine particular functional and pedagogical needs, establishing the project’s operational setting [24]. Building on this foundation, use cases and user stories were developed to translate extensive expectations into user-centred interaction scenarios. These findings were ultimately consolidated into formal requirements, categorising the platform’s core functions and behaviours (functional) alongside critical limitations, including performance, reliability, and usability (non-functional) [23, pp. 101–111].

2.4 System Design and Development Approach

The development process followed an iterative and incremental model, in which the system is constructed and refined through successive functional increments rather than a linear sequence [23, pp. 46, 50]. This approach supports the progressive validation of individual components, which is particularly important when integrating external AI services whose behaviour and output quality cannot be fully predicted in advance. Accordingly, core functionalities were implemented and tested in stages, enabling early verification and continuous refinement of features.

To complement this process, a layered architecture was adopted as a conceptual design principle, structuring the system into distinct layers with clearly defined responsibilities [23, pp. 177–178]. This separation of concerns enhances modularity and maintainability by ensuring that changes in one part of the system do not propagate across the entire structure. The architecture was therefore organised into logical layers, such as presentation, application logic, and data access, allowing independent development and modification.

The design process was further supported by the application of the Unified Modelling Language (UML), which provided a standardised method to represent the platform’s structure and behaviour [25]. By abstracting complex system interactions into a visual model, UML facilitates

clear communication and ensures a consistent understanding of how different components interact. In this context, UML modeling served as a conceptual blueprint, outlining the system architecture and guiding the implementation phase.

In addition, prototyping was employed to explore and refine the user interface design [26]. This approach enables early validation of design concepts and interaction workflows, reducing the risk of usability issues before full implementation. Through an iterative process of creating and improving interactive mock-ups using Figma, the interface structure, visual hierarchy, and functional flow were evaluated and optimised in a flexible and efficient manner.

2.5 Evaluation Methods

To assess the quality and correctness of the developed system, a combination of test cases, a Requirements Traceability Matrix (RTM), and heuristic evaluation was employed. This multi-method approach enables both functional verification and usability assessment, ensuring that the system meets its defined requirements while providing a coherent user experience.

The functional and non-functional requirements defined during the requirements engineering phase served as the basis for the evaluation process. To verify these requirements systematically, a set of test cases was specified prior to evaluating the final artifact. These test cases define the procedures and success criteria used to determine whether individual requirements are fulfilled, thereby making the evaluation process structured, repeatable, and transparent.

To consolidate this verification process, a RTM was used to systematically connect requirements, corresponding test cases, and evaluation outcomes throughout the development lifecycle [27]. By linking requirements to the test cases used for their verification, the RTM supports traceability and helps ensure that all defined requirements are consistently addressed [28]. In this project, the RTM was applied to validate that the implemented, AI-assisted functionalities align with the requirements defined in Section 3.1.4.

In addition to functional verification, a heuristic evaluation was conducted to assess the usability of the system interface [29]. This method involves a structured inspection of the system based on established usability principles, allowing potential issues to be identified without requiring external test participants. It was selected as an efficient and suitable approach for evaluating a functional prototype within the scope of this work, where the focus lies on identifying interface-related problems rather than measuring user performance.

The evaluation was conducted as an expert evaluation by the author, applying Nielsen's ten usability heuristics [29, p. 30] to the operational prototype. The analysis focused on key interaction processes, such as document handling and the use of AI-supported features, with the goal of identifying deviations from established design principles. Identified usability friction points were systematically documented and classified according to their severity, enabling a structured prioritisation of necessary improvements.

3 Solution

The following chapter presents the complete solution, including the derived requirements, system design, implementation, and evaluation results.

3.1 Requirements and Analysis Results

The solution development begins with a structured requirements analysis, starting from stakeholder considerations and progressively refining functional and non-functional system requirements.

3.1.1 Stakeholder Analysis

Alovre primarily targets L2 learners as the main users who interact directly with the platform to access AI-assisted reading support. Language teachers act as secondary stakeholders by influencing adoption in educational contexts, while system developers and external AI service providers define the technical implementation and constraints of the system.

3.1.2 User Stories

User stories outline the key user interactions with the system from a goal-oriented perspective.

- **US1:** Upload personal reading documents.
- **US2:** Open and continue previously uploaded texts.
- **US3:** Request translation and explanation of selected words.
- **US4:** Generate images for selected words to support understanding.
- **US5:** Listen to the pronunciation of selected words.

3.1.3 Use Cases

Based on the stakeholder analysis and as outlined in the Requirements Engineering Approach (Section 2.3), a set of core use cases was defined to represent the primary interactions between the user and the system. These use cases describe how the main functionalities of the platform are accessed and utilised from a user perspective.

The identified use cases focus on the central workflow of AI-assisted reading, including document management and the use of multimodal support features such as translation, explanation, image generation, and TTS.

Table 2 provides an overview of the key use cases.

Table 2: Overview of use cases.

Title	Actor(s)	Goal / Description	Preconditions	Main Flow
Document Upload	L2 Learner	Upload a new document or text for AI-assisted reading	User is on the upload page	1. Enter a title. 2. Select language. 3. Upload or paste text. 4. Confirm upload. 5. System processes and stores document.
Open Document	L2 Learner	Open an existing document to continue reading	User has uploaded documents	1. Open dashboard. 2. View documents. 3. Select document. 4. System loads reader.
Translation and Explanation	L2 Learner	Get translation (L1) and explanation (L2)	Document is open	1. Select word. 2. Trigger function. 3. Request sent to AI. 4. Result displayed.
Image Generation	L2 Learner	Generate visual representation	Document is open	1. Select word. 2. Trigger image generation. 3. Request sent. 4. Image displayed.
TTS	L2 Learner	Listen to pronunciation	Document is open	1. Select word. 2. Trigger TTS. 3. Request sent. 4. Audio played.

3.1.4 Functional and Non-Functional Requirements

The requirements emphasise document-based reading workflows combined with multimodal AI assistance at the word level. In particular, translation, explanation, image generation, and TTS functionality form the core features of the system, while usability, performance, and maintainability define the key quality constraints for Alovre.

Table 3: Functional requirements of the AI-assisted reading system.

ID	Requirement	Description
FR1	Document Upload	The system shall allow users to upload reading documents (e.g., PDF files) for processing and storage in the reading library.
FR2	Document Selection	The system shall provide a library interface where users can view and select previously uploaded documents.
FR3	Text Display	The system shall render uploaded documents in a readable format within the reading interface.
FR4	Word Selection	The system shall allow users to select or highlight individual words within the displayed text.
FR5	Translation and Explanation	The system shall generate a translation in the user's native language and an explanation in the target language for selected words.
FR6	Image Generation	The system shall generate and display a contextual image for selected words upon user request.
FR7	TTS Output	The system shall generate and play audio pronunciation for selected words using a TTS service.
FR8	AI Service Integration	The system shall communicate with external AI services via Application Programming Interfaces (APIs) to provide explanations, image generation, and audio synthesis.

Table 4: Non-functional requirements of the AI-assisted reading system.

ID	Requirement	Description
NFR1	Usability	The interface shall minimise interaction steps required to access AI-assisted reading features.
NFR2	Performance	AI-generated outputs (text, image, audio) shall be delivered within acceptable response times.
NFR3	Reliability	The system shall handle failed external AI service requests gracefully and inform the user appropriately.
NFR4	Scalability	The architecture shall support the integration of multiple external AI services without structural redesign.
NFR5	Maintainability	The system shall follow a layered architecture to ensure modularity and ease of maintenance.
Continued on next page...		

Continuation of Table 4		
ID	Requirement	Description
NFR6	Compatibility	The system shall be accessible via modern web browsers without additional software installation.

3.2 System Overview and Functionality

Alovre is a web-based application that supports L2 learning through AI-assisted interaction with user-provided texts. Users can upload documents (PDF, TXT) or input raw text via the interface, after which the content is processed, including text extraction and normalisation, to enable interactive usage.

All content is managed in a centralised library where users can view, edit, or delete documents. Selecting a document opens the main reading interface, which consists of a text display area and a side panel providing access to AI-based assistance features.

Within the reading interface, individual words are selectable. On the basis of a selected word, the system offers three core functionalities: (1) contextual translation and explanation generated via a language model, (2) TTS output for auditory support, and (3) context-based image generation to support visual understanding. These features are integrated into a unified workflow, with the backend orchestrating multiple AI services to generate responses.

Additionally, the system stores previously generated outputs, allowing users to revisit translations, audio, and images without repeated processing. Overall, the application combines textual, auditory, and visual assistance to create an interactive and supportive reading environment.

3.3 User Interface

The user interface is designed to provide a visually calm and intuitive reading environment, with a strong focus on readability and low cognitive load. A consistent layout structure is used throughout the application, with a fixed top navigation bar enabling access to the main areas (homepage, upload, library).

The visual design follows a warm, low-contrast colour scheme (beige backgrounds and brown text) to reduce eye strain during extended reading sessions. Interactive elements such as buttons and icons are highlighted using a distinct accent colour, while an optional dark mode provides an alternative theme with reduced screen glare. The interface further employs soft, rounded shapes to create a clear and approachable visual structure.

The final interface design was preceded by an iterative prototyping phase using AI-supported mock-up generation. Early interface concepts and their evolution into the implemented frontend are documented in Appendix C.

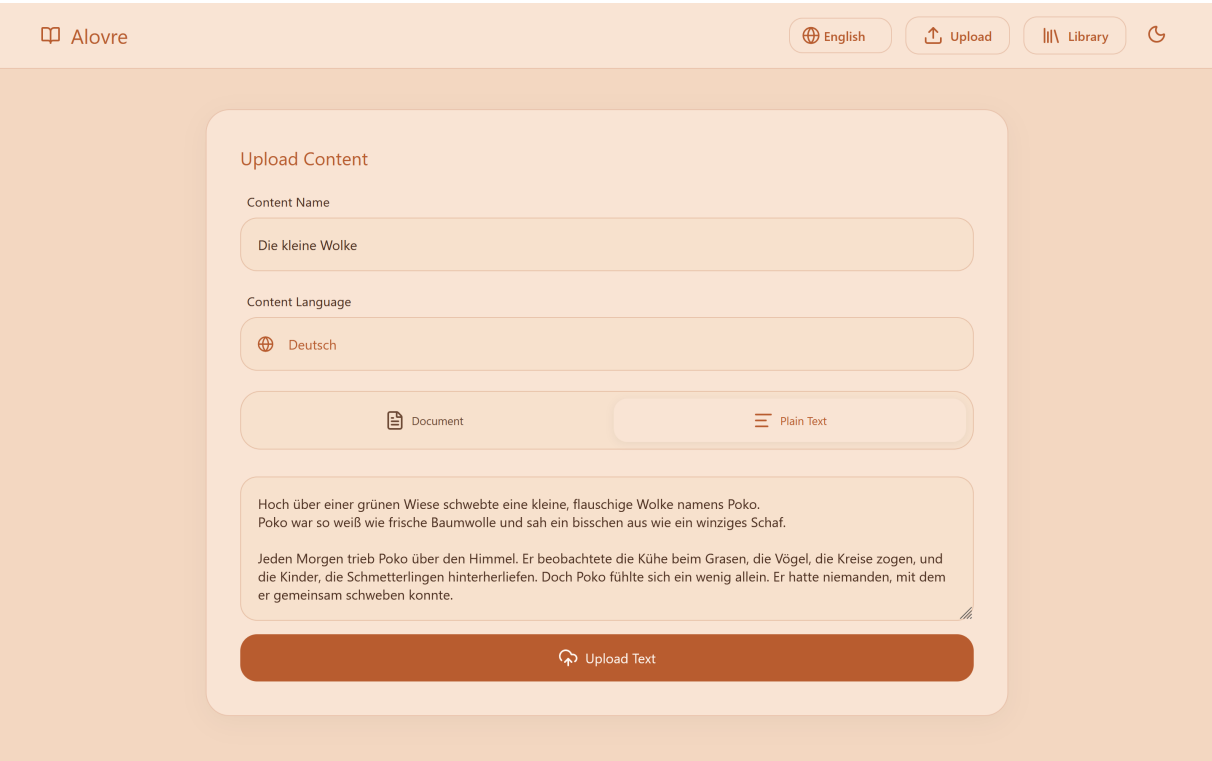


Figure 1: Alovre—upload interface.

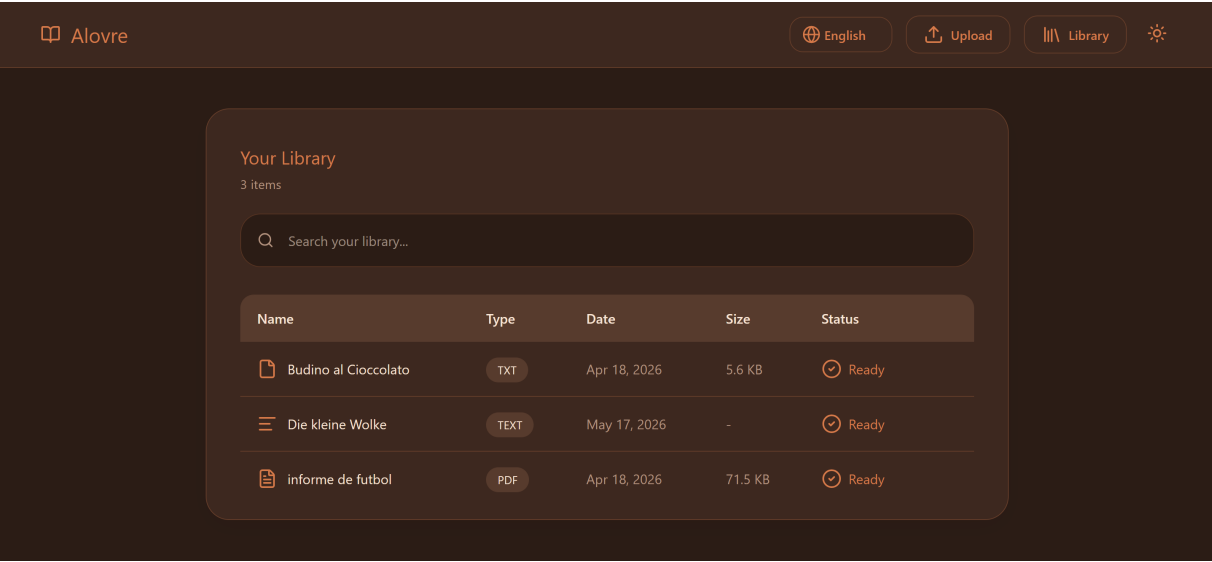


Figure 2: Alovre—library interface.

The central reading interface is organised as a dual-panel layout, with the text displayed on the left and an interaction panel on the right. The interface emphasises direct manipulation: words are selectable, and available actions are visually indicated through context-aware controls. Icons are used as a primary means of interaction, reducing textual complexity. For example, symbols for translation, audio playback, and image generation provide immediate recognition of available features and are consistently styled to guide user attention.

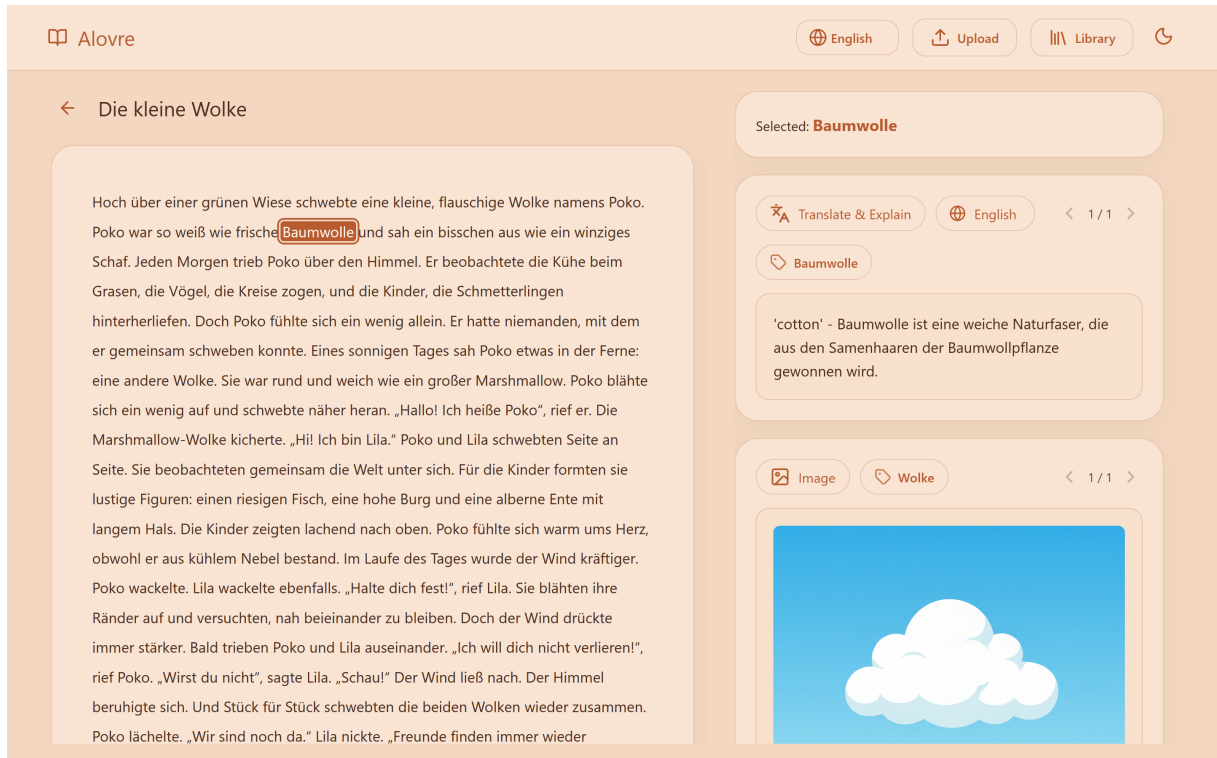


Figure 3: Alovre—reading interface.

Additional interface elements, such as hover effects, disabled button states, and non-intrusive notifications, provide immediate feedback without interrupting the reading flow. In general, the interface design prioritises clarity, consistency, and efficient interaction, enabling users to focus on reading while seamlessly accessing multimodal support features.

3.4 Software Architecture

Alovre utilises a layered design to decouple user interaction, application logic, data management, and external services. This structure ensures a separation of concerns while enhancing maintainability, scalability, and extensibility. An overview of the architecture and its main components is illustrated in Figure 4.

Organised into two primary tiers, the platform comprises a frontend and a backend. The frontend serves as the presentation layer, managing user engagement and data visualisation.

By communicating with the backend via HTTP [30] requests, it grants users access to the full range of system features.

To manage information effectively, the architecture differentiates between structured and unstructured content. A relational database stores metadata, whereas an object storage system hosts uploaded files and generated media. This dual approach facilitates the efficient processing of large binary assets alongside relational records.

AI capabilities are incorporated as external services, reachable only through the backend. This central server acts as a unified interface that prepares requests, selects suitable models, and processes results. Consequently, the frontend only engages with high-level functions, keeping the underlying AI infrastructure completely abstracted.

The entire system is deployed as a collection of independent services within a shared environment. This setup promotes modularity and ensures consistent performance across various platforms.

Component Diagram - Simple Container Architecture

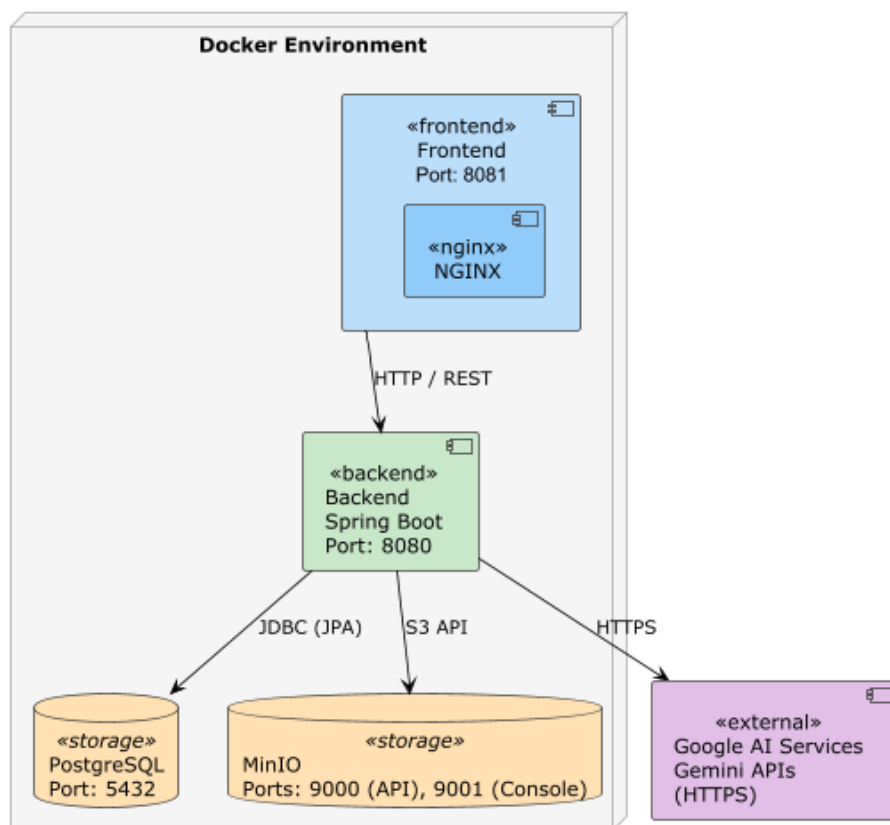


Figure 4: Alovre—UML component diagram.

3.5 Implementation

The implementation of the system is based on a defined technology stack and is subsequently detailed across backend, frontend, processing, and deployment components.

3.5.1 Technology Stack

The prototype was implemented using a modern web technology stack selected for reliability, ease of development, and compatibility with containerised deployment.

On the backend side, Java 21 [31] and Spring Boot 3.5.12 [32] to implement the Representational State Transfer (REST)-based [33] application logic and service integration. Maven 3.9.6 [34] served as the build and dependency management tool. In addition, Tesseract OCR 5 [35] was integrated for text extraction from scanned PDF documents.

For data persistence, PostgreSQL 15 [36] was used as the relational database, while MinIO [37] served as S3-compatible object storage for uploaded documents and generated media files.

The frontend was realised using HTML [38], CSS [39], and vanilla JavaScript [40], served through an Nginx [41] web server. This lightweight approach reduced implementation complexity while remaining sufficient for the required interactive user interface.

Deployment and local execution were managed through Docker [42] and Docker Compose 3.9 [43], allowing all components to run as isolated but connected services.

AI functionality was integrated via the Google Gemini SDK 1.44.0 [44], providing access to language, image-generation, and TTS models.

Development was supported by IntelliJ IDEA Ultimate [45], GitHub [46] for version control, Docker Desktop [47] for container management, and AI-assisted tools such as ChatGPT [48], Gemini [49], Figma Make [50], and Junie [51].

3.5.2 Backend Implementation

The Alore backend was implemented as a Spring Boot application and functions as the central coordination unit of the system. It exposes REST interfaces, manages business logic, communicates with the database and object storage, and integrates external AI services. To ensure maintainability and a clear separation of concerns, the backend is structured into dedicated packages following a layered architecture.

At the infrastructure level, the `config` package contains global application settings and reusable beans. This includes asynchronous task execution [52], MinIO client initialisation, Optical Character Recognition (OCR) configuration, and Cross-Origin Resource Sharing [53] policies that enable secure communication between frontend and backend. By isolating such concerns, technical configuration remains independent from business logic.

Building on this foundation, the `controller` package provides the REST API layer through which all client requests are processed. Two main controllers were implemented: `AiController` and `ContentController`. The AI controller offers endpoints for contextual explanation and translation, image generation, and TTS synthesis. In contrast, the content controller manages document upload, manual text input, retrieval of stored documents, metadata updates, deletion, and streaming of generated media files.

The persistent data structures of the application are defined in the `model` package. Its central entity is `Content`, which represents uploaded reading material. Additional entities such as `Annotation`, `Image`, and `Audio` store AI-generated outputs and maintain references to their corresponding content entries.

To access these entities efficiently, the `repository` package implements the data access layer using Spring Data JPA [54]. Repository interfaces provide automatically generated Create, Read, Update, and Delete operations, which significantly reduce manual SQL development while enabling structured and type-safe access to PostgreSQL.

The core application logic is located in the `service` package. Here, `ContentService` manages uploads and asynchronous document processing, while `AiService` orchestrates requests to Google Gemini for text, image, and audio generation. Furthermore, `StorageService` handles MinIO object storage, whereas `PdfService` and `OcrService` implement text extraction from PDF files and scanned documents. Supporting utility classes additionally perform tasks such as converting generated audio into browser-compatible formats.

Communication between frontend and backend is standardised through the `dto` package, which contains Data Transfer Objects for requests and responses. This approach separates internal domain models from externally exposed API structures.

Several Spring Boot features further supported the implementation, including dependency injection [55], embedded server deployment, Bean Validation [56], asynchronous execution, Actuator monitoring [57], and automatic repository generation. Together, these mechanisms enabled a robust and maintainable backend capable of coordinating storage, document processing, and external AI services efficiently.

3.5.3 Frontend Implementation

The Alore frontend, constructed with HTML5, CSS3, and vanilla JavaScript, follows a lightweight multi-page architecture utilising reusable shared components. By eschewing modern JavaScript frameworks in favor of modular scripts, native browser APIs, and direct Document Object Model [58] manipulation, the design reduces overall complexity while ensuring responsive interactions and a clear separation between pages.

At the core of this implementation lies the `common.js` module, which centralises global functions such as navigation loading, theme toggling, icon rendering, and multilingual support. To maintain a consistent user interface and eliminate redundant markup, the navigation bar is dynamically injected into every page upon loading.

Internationalisation is handled through a centralised translation object supporting multiple languages. Rather than reloading the page, JavaScript updates text elements, placeholders, and labels at runtime, storing the user's preference in the browser's local storage. Whenever the language is changed, custom events are triggered to ensure that page-specific modules refresh dynamic content immediately.

Persistent user preferences, such as dark mode, are similarly managed via browser storage. These visual themes are applied by toggling CSS classes on the root document element, while the Lucide [59] icon library is reinitialised following any dynamic updates to guarantee correct rendering.

The upload system facilitates two distinct input modes: file uploads and direct text entry. Before any data reaches the backend, JavaScript validation verifies required fields, file types, and general input integrity. Once validated, the system transmits either `FormData` [60] or JSON [61] payloads asynchronously using the Fetch API [60], keeping the user informed through toast notifications and visual loading indicators.

Functioning as a dynamic dashboard, the library management module provides an interface for stored documents. It periodically polls the backend for metadata, updating the table only when changes occur, and enables client-side search, edit, and delete functions. To maintain data integrity, documents currently undergoing processing are visually marked and disabled until extraction is finished.

The most interactive element of the frontend is the reader mode. In this module, uploaded text is tokenised into selectable word elements; clicking a word activates specific AI tools for translation, image generation, and pronunciation. These requests are sent asynchronously to the backend alongside contextual sentence data, with the results rendered immediately in a side panel. Furthermore, session-based tool states allow users to cycle through previously generated outputs.

Ultimately, the frontend implementation emphasises simplicity, rapid load times, and direct interaction via native web technologies. This strategy resulted in a responsive reading environment featuring persistent settings and dynamic AI-assisted tools without the overhead of heavyweight frameworks.

3.5.4 Document Processing Pipeline

Central to the technical architecture of Alovre is the document processing pipeline, which transforms raw uploaded material into accessible text for the interactive interface. The system is designed to handle a variety of sources, specifically plain text input, text files, and PDF documents.

At the frontend level, users interact with separate modes to choose between direct text entry and file uploads. To maintain data integrity, JavaScript validation confirms that all mandatory fields are populated before submission. While the interface supports both manual selection and drag-and-drop interactions, any invalid file types are immediately rejected. These upload re-

quests are transmitted asynchronously, allowing loading notifications to provide real-time feedback throughout the processing stage.

Once the data reaches the backend, it is managed by dedicated REST endpoints within the `ContentController`. The system distinguishes between transmission methods, handling file uploads as multipart requests containing metadata—such as title and language—while processing manual text entries as JSON payloads.

For file-based submissions, the `StorageService` first persists the original document in MinIO object storage. A unique Universally Unique Identifier-based [62] key is assigned to each object to resolve potential filename conflicts and ensure reliable retrieval. Following successful storage, a `Content` entity is created in the PostgreSQL database to track metadata, including the title, language, `file type`, size, timestamp, and storage reference.

To enhance the perceived speed of the application, documents appear in the library immediately after upload. The actual text extraction is then executed in the background using Spring's `@Async` mechanism, preventing the user interface from locking up.

When handling PDF files, the pipeline first attempts direct text extraction using Apache PDF-Box [63]. Should this fail to yield usable content, the system automatically triggers a fallback to Tesseract OCR, providing robust support for scanned or image-based documents. In contrast, text files are read directly from storage as plain text, while manually entered input requires no further preprocessing.

Upon completion of these steps, the extracted content is written to the `textContent` field and the document's status flag is updated. By periodically requesting metadata updates, the frontend can detect exactly when a document has transitioned from processing to a ready state.

By integrating frontend validation, asynchronous workflows, object storage, and OCR capabilities, the implemented pipeline creates a cohesive workflow for heterogeneous input. This comprehensive approach ensures reliable data handling while preserving a smooth and responsive user experience.

3.5.5 AI Service Orchestration

A central technical component of Alovre is the orchestration of multiple AI capabilities through a unified backend service. Instead of exposing external model APIs directly to the frontend, all AI-related requests are routed through the `AiController` and coordinated by the dedicated `AiService`. This architecture separates user interaction from provider-specific implementation details and enables consistent request handling, prompt construction, persistence, and error control.

Three task-specific AI functions were integrated: contextual translation and explanation, image generation, and TTS synthesis. Each function is exposed through a dedicated REST endpoint (`/explain`, `/image`, and `/tts`) and internally mapped to a suitable Gemini model variant.

Contextual translation and explanation are handled via `gemini-2.5-flash-lite` [64].

When a user selects a word in reader mode, the frontend sends the selected term, document identifier, surrounding sentence context, and desired target language to the backend. Based on this data, the backend dynamically constructs a structured prompt requesting both a context-aware translation and a short explanation. The model is instructed to return JSON output, which is parsed and transformed into a standardised response object. Generated results are additionally stored as `Annotation` entities linked to the corresponding `Content` item.

Image-based assistance is implemented using `gemini-2.5-flash-image` [64]. Prompt generation focuses on producing clear and educational images that represent the selected word according to its textual context. Ambiguous terms are therefore disambiguated using the surrounding sentence before image generation is requested. The returned binary image data is extracted from the API response, uploaded to MinIO object storage, and persisted as an `Image` entity containing the storage reference and metadata.

Pronunciation generation relies on `gemini-2.5-flash-preview-tts` [64]. The backend creates a concise pronunciation prompt for the selected word and requests audio output using a predefined synthetic voice configuration. Returned audio is converted into a browser-compatible format to ensure compatibility before being stored in MinIO. Metadata such as the selected word, voice, and timestamp are then saved as an `Audio` entity.

A major advantage of this orchestration layer is that model selection logic remains fully encapsulated within the backend. The frontend only requests functional actions such as translation, image generation, or pronunciation, while the server determines which model, configuration, response modality, and post-processing steps are required. This reduces frontend complexity and allows future model replacements without changes to the user interface.

The implementation also standardises AI output handling across modalities. Text results are returned as structured JSON responses, while generated media files are stored centrally and referenced through object keys. If generation fails, backend exceptions lead to HTTP error responses that are handled in the frontend through notifications and disabled loading states.

This service-oriented orchestration approach enabled the integration of heterogeneous AI models into one coherent learning application while preserving maintainability, extensibility, and a consistent user experience.

3.5.6 Deployment and Development Environment

Deployment of Alovre is based on Docker Compose, which orchestrates all components of the application in a containerised and reproducible environment. This approach ensures that the complete stack can be started consistently across different machines with minimal configuration effort.

Structurally, the architecture is composed of four services: a Spring Boot backend, an Nginx-based frontend, a PostgreSQL database, and a MinIO object storage instance. All services communicate through a shared Docker network and are addressed via container hostnames. For instance, the backend accesses PostgreSQL via `postgres` and MinIO via `minio`, which

removes the need for manual network configuration and simplifies service integration.

In terms of orchestration, Docker Compose manages service startup order and configuration. Configuration details such as database credentials, storage access keys, and external API keys are provided via environment variables [65], ensuring a clear separation between configuration and application logic.

As a result, this setup enables fully reproducible local development and deployment using a single command while maintaining a production-like system architecture. Finally, version control is managed via Git [66] and GitHub, supporting structured development workflows and full traceability of changes throughout the project lifecycle.

3.6 Evaluation Results

This section presents the results of the systematic evaluation of Alovre through requirement verification and heuristic usability inspection.

3.6.1 Requirements Traceability Matrix

The verification process was based on a set of functional and non-functional test cases covering the core workflows and system constraints of Alovre.

Table 5 summarises the functional test cases related to document handling and multimodal AI-assisted interaction.

Table 5: Functional test cases.

Test ID	Procedure (Action)	Success Criteria
TC-FR1	Upload a valid PDF document through the upload interface.	The document is accepted, stored, and available in the system.
TC-FR2	Open the dashboard and select a previously uploaded document.	The selected document loads correctly in the reading interface.
TC-FR3	Open a document and view its content.	The text is displayed in a readable and correctly formatted manner.
TC-FR4	Select a word in the displayed text.	The word is highlighted or visually marked.
TC-FR5	Request translation and explanation for a selected word.	Translation and explanation are displayed correctly.
TC-FR6	Request an image for a selected word.	A relevant contextual image is generated and displayed.
Continued on next page...		

Continuation of Table 5		
Test ID	Procedure (Action)	Success Criteria
TC-FR7	Request audio pronunciation for a selected word.	Audio playback corresponds to the selected word.
TC-FR8	Trigger multiple AI-based features.	The system communicates with external AI services and returns valid outputs.

Table 6 presents the non-functional test cases focusing on usability, performance, reliability, and system robustness.

Table 6: Non-functional test cases.

Test ID	Procedure (Action)	Success Criteria
TC-NFR1	Perform a typical user flow (word selection and AI request).	AI features are accessible with minimal steps and without confusion.
TC-NFR2	Trigger multiple AI requests.	Responses are returned within an acceptable time frame (approx. 2–9 seconds).
TC-NFR3	Simulate AI service failure.	The system displays an error message and remains stable.
TC-NFR4	Integrate multiple AI services.	The system supports integration without structural changes.
TC-NFR5	Review system structure.	The architecture is modular and maintainable.
TC-NFR6	Run the system in different web browsers.	The system functions correctly without additional installation.

The RTM in Table 7 consolidates the evaluation by linking the functional and non-functional requirements defined in Section 3.1.4 to their corresponding test cases and verification outcomes, providing a consolidated view of requirement fulfilment.

Table 7: Requirements traceability matrix.

ID	Requirement	Test Case(s)	Result
FR1	Document Upload	TC-FR1	Passed
FR2	Document Selection	TC-FR2	Passed
FR3	Text Display	TC-FR3	Passed
Continued on next page...			

Continuation of Table 7			
ID	Requirement	Test Case(s)	Result
FR4	Word Selection	TC-FR4	Passed
FR5	Translation and Explanation	TC-FR5	Passed
FR6	Image Generation	TC-FR6	Passed
FR7	TTS Output	TC-FR7	Passed
FR8	AI Service Integration	TC-FR5, TC-FR6, TC-FR7, TC-FR8	Passed
NFR1	Usability	TC-NFR1	Passed
NFR2	Performance	TC-NFR2	Passed
NFR3	Reliability	TC-NFR3	Partially Met
NFR4	Scalability	TC-NFR4	Passed
NFR5	Maintainability	TC-NFR5	Passed
NFR6	Compatibility	TC-NFR6	Passed

The RTM results show that all functional requirements were successfully fulfilled, confirming the correct implementation of the core system features and the successful integration of multimodal AI-assisted functionality within the reading environment.

The non-functional requirements were also largely satisfied. The evaluation indicates that the system provides a usable, responsive, scalable, and maintainable foundation for AI-assisted L2 reading. One limitation was identified in reliability (NFR3), where error handling for external AI service failures remains limited. While the system maintains stability, it currently provides only generic feedback in such cases.

Therefore, the results demonstrate that Alovre achieves its intended functionality, with reliability improvements representing the primary area for future refinement.

3.6.2 Heuristic Evaluation Results

The system interface was evaluated against Nielsen’s usability heuristics, focusing on core interaction workflows such as document handling and AI-assisted features. Identified usability issues were classified using Nielsen’s severity rating scale [29, p. 47], enabling a structured prioritisation of improvements.

Detailed evaluation results for each use case are provided in Appendix E. The following summarises the key findings across all evaluated workflows.

The heuristic evaluation across all system modules—Document Upload, Open Document, Translation and Explanation, Image Generation, and TTS—reveals a generally consistent and functionally robust user experience. Across all evaluated workflows, the system demonstrates

strong alignment with core usability principles, particularly in the areas of error prevention, system consistency, and recognition-based interaction design.

A key strength observed across all use cases is the consistent interaction model for AI-assisted features. Users interact with a unified pattern for requesting translations, generating images, and triggering audio output, which significantly reduces the learning effort and supports predictable system behaviour. In addition, most workflows provide effective visual feedback mechanisms and maintain stable navigation structures, contributing to overall system clarity and usability.

Despite these strengths, several recurring usability limitations were identified across multiple components. The most prominent issue concerns interaction efficiency, particularly in workflows that require repeated AI requests. In these cases, the distance between the selected text and the AI action controls increases interaction effort during extended reading sessions. Additional interaction mechanisms such as keyboard shortcuts or action buttons placed closer to the selected text could further improve efficiency and reduce repetitive cursor movement.

Another common limitation concerns contextual continuity. While the system persistently displays the currently selected word within the interface, generated outputs are still not linked to the exact occurrence or position of the word within the source text. This can become problematic when identical words appear multiple times in different contexts. Stronger visual associations between text selections and generated outputs could further improve contextual orientation.

A third consistent finding concerns system feedback and responsiveness. Although all modules provide functional loading indicators, these are often implemented as global notifications rather than localised, component-based feedback. This reduces perceived system transparency, especially in cases of variable AI response times or failed external service requests.

Beyond these issues, the system's handling of content history introduces further usability constraints. Although previously generated translations, images, and audio outputs remain accessible through navigation controls, users cannot selectively remove individual entries. Introducing more flexible history management options could improve interaction control and overall usability during longer reading sessions.

Finally, error recovery mechanisms are generally limited across all tools. While the system maintains stability under failure conditions, user-facing recovery options such as retry actions or inline error resolution are not consistently implemented.

Overall, the evaluation confirms that the system provides a coherent and usable foundation for AI-assisted reading. All identified issues fall within minor to moderate severity levels, with no critical usability violations detected. Quantitative analysis of the findings (see Appendix E) results in a mean severity score of 0.54, categorising the identified limitations primarily as cosmetic issues. The main opportunities for improvement lie in enhancing interaction efficiency, strengthening contextual persistence, and improving localised system feedback mechanisms.

4 Discussion

This chapter critically reflects on the developed system, evaluating its limitations, overall outcomes, and lessons learned during the development process.

4.1 Critical Reflection and Limitations

While Alovre demonstrates the feasibility of integrating multimodal AI services into a unified reading platform, several limitations remain.

Interaction within the system requires multiple manual steps, as users must first select a word and then explicitly trigger actions via interface buttons. More efficient interaction patterns, such as action buttons or keyboard shortcuts, are not implemented, and could improve usability.

Reliance on external AI services introduces challenges regarding cost, scalability, and data privacy, as user content is processed by third-party APIs. These issues could be mitigated by integrating locally hosted, open-source models.

The system is intentionally designed as a supportive reading tool rather than a full language-learning platform. It assists with vocabulary comprehension but does not cover grammar or foundational language learning. Consequently, it is best suited for users with an existing level of language proficiency and provides no benefit for individuals without reading ability.

The prototype is particularly suited for continuous prose such as stories, articles, or simplified reading material. The current implementation primarily supports plain text, meaning that structural elements such as tables, graphics, or formulas are not preserved during preprocessing. As a result, highly structured or domain-specific documents, including academic publications, technical manuals, or scientific texts, are less suitable for the system. In addition, generated outputs are not always unambiguous. Especially for abstract terms or specialised concepts, generated images may require interpretation and are often most effective when combined with textual explanations.

A quantitative evaluation of AI output quality (e.g., translation accuracy or image correctness) was not conducted, as these aspects depend on external model performance and are often difficult to evaluate objectively. Instead, the evaluation focuses on system functionality, usability, and requirement fulfilment.

These limitations highlight potential areas for future improvement, including enhanced document processing, improved interaction design, and more independent AI integration.

4.2 Fulfilment of Research Goals

The research question stated in Section 1.3 asked how a multimodal AI-assisted reading system can be designed and implemented to support L2 learners, and to what extent such a system is usable.

The design aspect of the research question is addressed through the Requirements and Analysis Results (Section 3.1), where stakeholder analysis, user stories, use cases, and functional and non-functional requirements were used to derive the conceptual foundation of the system. Building on these findings, the proposed architecture and interaction concepts were developed to support multimodal AI-assisted reading workflows.

The implementation aspect is demonstrated through the technical realisation of the system, spanning from System Overview and Functionality (Section 3.2) to Implementation (Section 3.5). The resulting prototype successfully integrates multiple AI-supported functionalities within a unified reading environment. The evaluation results further show that the majority of functional requirements were fulfilled, with only minor limitations remaining in the area of error handling.

The usability aspect of the research question is addressed through the Evaluation Results (Section 3.6), particularly the heuristic evaluation presented in Section 3.6.2. The evaluation yielded a low average severity rating of 0.54, indicating that the system provides a stable and user-friendly interaction experience despite minor usability limitations. The findings further demonstrate that the implemented interaction patterns and multimodal support features are generally consistent, understandable, and suitable for AI-assisted reading workflows.

Overall, the objective of developing a functional, multimodal AI-assisted reading prototype following the DSR methodology was achieved. The resulting system demonstrates that a platform such as Alovre can be effectively designed using a layered and modular architecture while providing a coherent and usable environment for AI-supported L2 reading.

4.3 Generalisation and Transferability

The developed system is broadly applicable to a wide range of users, as it is designed as a general-purpose reading support tool. Its intuitive and minimal user interface enables accessibility across different age groups, from younger to older users, without requiring prior technical expertise.

However, the applicability is inherently limited to individuals with basic reading ability, as the system focuses on supporting comprehension rather than teaching literacy.

As a prototype developed and evaluated by a single author, the system has not yet been validated in diverse real-world contexts. Future evaluations with a broader user base would be necessary to further assess usability and effectiveness. Nevertheless, the underlying concept and architecture are transferable and can be adapted to different user groups, languages, and learning scenarios with minimal modification.

4.4 Retrospective and Lessons Learned

The development of Alovre provided several insights regarding both technical implementation and the complete research process. The use of AI significantly accelerated development and writing, often improving productivity and quality. However, AI-generated outputs were not always reliable, occasionally containing inaccuracies or inconsistencies in both code and text. This required continuous, critical evaluation and manual refinement, highlighting that AI should be used as a supportive tool rather than a fully autonomous solution.

From a technical perspective, the importance of clear system structure and modular design became evident. The layered architecture and separation of concerns simplified development, debugging, and integration of external services. At the same time, working with multiple components (frontend, backend, storage, and AI services) introduced complexity, particularly in coordinating data flow and ensuring stable communication between services.

Another key lesson concerns scope management. Focusing on a well-defined core functionality—namely AI-supported reading—proved essential to achieving a working prototype within limited time. Features such as richer document formatting or fully optimised interaction design were deliberately left out to maintain feasibility.

In general, the project demonstrates that combining modern web technologies with AI services is highly effective, but requires careful design decisions, critical thinking, and a clear prioritisation of goals throughout the development process.

Bibliography

- [1] N. L. Sangers, L. Van Der Sande, C. Welie, M. Dobber, and R. Van Steensel, "Learning a Language through Reading: A meta-analysis of studies on the effects of extensive reading on second and foreign language learning," *Educational Psychology Review*, vol. 37, no. 4, Sep. 2025, doi: 10.1007/s10648-025-10068-6.
- [2] A. Pellicer-Sánchez, "INCIDENTAL L2 VOCABULARY ACQUISITION FROM AND WHILE READING," *Studies in Second Language Acquisition*, vol. 38, no. 1, pp. 97–130, Aug. 2015, doi: 10.1017/s0272263115000224.
- [3] T. Zhu, Y. Zhang, and D. Irwin, "Second and Foreign Language Vocabulary Learning through Digital Reading: A Meta-Analysis," *Education and Information Technologies*, vol. 29, no. 4, pp. 4531–4563, Jul. 2023, doi: 10.1007/s10639-023-11969-1.
- [4] M. Pigada and N. Schmitt, "Vocabulary acquisition from extensive reading: A case study," *Reading in a Foreign Language*, vol. 18, no. 1, pp. 1–28, Apr. 2006, doi: 10.64152/10125/66611.
- [5] N. Schmitt, X. Jiang, and W. Grabe, "The Percentage of Words Known in a Text and Reading Comprehension," *The Modern Language Journal*, vol. 95, no. 1, pp. 26–43, Feb. 2011, doi: 10.1111/j.1540-4781.2011.01146.x.
- [6] R. E. M. P. Chandrika and R. Ambedkar, "The effect of cognitive load on second language acquisition," *International Journal of Humanities and Social Science Invention*, vol. 9, no. 2, pp. 102–107, Feb. 2020, ser. I.
- [7] Q. Liu, "Reading Anxiety in First and Second Language Acquisition: A Comprehensive Literature review," *Journal of Modern Social Sciences*, vol. 2, no. 3, Jun. 2025, doi: 10.71113/jmss.v2i3.329.
- [8] H. Yuan, "Artificial intelligence in language learning: biometric feedback and adaptive reading for improved comprehension and reduced anxiety," *Humanities and Social Sciences Communications*, vol. 12, no. 1, Apr. 2025, doi: 10.1057/s41599-025-04878-w.
- [9] C. K. Lo, P. L. H. Yu, S. Xu, D. T. K. Ng, and M. S.-Y. Jong, "Exploring the application of ChatGPT in ESL/EFL education and related research issues: a systematic review of empirical studies," *Smart Learning Environments*, vol. 11, no. 1, Nov. 2024, doi: 10.1186/s40561-024-00342-5.
- [10] Z. Zhang and X. Huang, "The impact of chatbots based on large language models on second language vocabulary acquisition," *Heliyon*, vol. 10, no. 3, e25370, Feb. 2024, doi: 10.1016/j.heliyon.2024.e25370.

- [11] M. Kořak-Babuder, J. Kormos, M. Ratajczak, and K. Piřorn, "The effect of read-aloud assistance on the text comprehension of dyslexic and non-dyslexic English language learners," *Language Testing*, vol. 36, no. 1, pp. 51–75, Feb. 2018, doi: 10.1177/0265532218756946.
- [12] H. Shafiee Rad, "Reinforcing L2 reading comprehension through artificial intelligence intervention: refining engagement to foster self-regulated learning," *Smart Learning Environments*, vol. 12, 2025, art. no. 23, doi: 10.1186/s40561-025-00377-2.
- [13] Z. Wu and Y. Zhang, "The data visualization and intelligent text analysis for effective evaluation of English language teaching," *Scientific Reports*, vol. 15, no. 1, p. 22 737, Jul. 2025, doi: 10.1038/s41598-025-08182-0.
- [14] X. Zhang, V. Dörig, P. Cui, V. Zouhar, T. Netland, and M. Sachan, "Harmonizing Assistance: Moderating Visual and Textual Aids in AI-Enhanced Textbook Reading with IRead," *International Journal of Artificial Intelligence in Education*, vol. 35, no. 6, pp. 3780–3812, Nov. 2025, doi: 10.1007/s40593-025-00515-4.
- [15] LingQ. "Lingq subscription plans," Accessed: Apr. 10, 2026. [Online]. Available: <https://www.lingq.com/de/accounts/subscription/?source=upgradeButton>
- [16] Readlang. "Readlang pricing," Accessed: Apr. 10, 2026. [Online]. Available: <https://readlang.com/pricing>
- [17] Microsoft. "Microsoft edge immersive reader," Accessed: Apr. 10, 2026. [Online]. Available: <https://www.microsoft.com/de-de/edge/features/immersive-reader?form=MT0160>
- [18] LingoReader. "Lingoreader app," Accessed: Apr. 10, 2026. [Online]. Available: <https://lingoreaderapp.com/>
- [19] Microsoft Learn. "Microsoft edge browser policies - immersivereaderpicturedictionaryenabled," Accessed: Apr. 10, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/deployedge/microsoft-edge-browser-policies/immersivereaderpicturedictionaryenabled>
- [20] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A Design Science research Methodology for Information Systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, Dec. 2007, doi: 10.2753/mis0742-1222240302.
- [21] R. F. Baumeister and M. R. Leary, "Writing narrative literature reviews," *Review of General Psychology*, vol. 1, no. 3, pp. 311–320, Sep. 1997, doi: 10.1037/1089-2680.1.3.311.
- [22] B. A. Kitchenham and L. M. Pickard, "Evaluating software engineering methods and tools," *ACM SIGSOFT Software Engineering Notes*, vol. 23, no. 1, pp. 24–26, Jan. 1998, doi: 10.1145/272263.272268.
- [23] I. Sommerville, *Software Engineering*, 10th ed. Harlow, England: Pearson Education Limited, 2016.
- [24] R. Brugha, "Stakeholder analysis: a review," *Health Policy and Planning*, vol. 15, no. 3, pp. 239–246, Sep. 2000, doi: 10.1093/heapol/15.3.239.

- [25] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*. Addison-Wesley Professional, 1999.
- [26] S. Houde and C. Hill, “What do Prototypes Prototype?” In *Elsevier eBooks*, doi: 10.1016/b978-044481862-1.50082-0, 1997, pp. 367–381.
- [27] O. C. Z. Gotel and A. C. W. Finkelstein, “An analysis of the requirements traceability problem,” in *Proceedings of the 1st International Conference on Requirements Engineering*, doi: 10.1109/ICRE.1994.292398, Colorado Springs, CO, USA, 1994, pp. 94–101.
- [28] P. Bourque and R. E. Fairley, *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 3.0*, 3rd ed., P. Bourque and R. E. Fairley, Eds. Piscataway, NJ, USA: IEEE Computer Society, 2014, ISBN: 978-0-7695-5166-1.
- [29] J. Nielsen, “Heuristic evaluation,” in *Usability Inspection Methods*, J. Nielsen and R. L. Mack, Eds., John Wiley & Sons, 1994, pp. 25–62.
- [30] Fielding, R. and Nottingham, M. and Reschke, J. “Http semantics (rfc 9110).” Status: Internet Standard, IETF, Accessed: May 7, 2026. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9110.html>
- [31] Oracle. “Java platform, standard edition (java se) 21 documentation.” Java SE 21 (LTS), Oracle Corporation, Accessed: May 7, 2026. [Online]. Available: <https://docs.oracle.com/en/java/javase/21/>
- [32] Broadcom. “Spring boot documentation.” Version 3.x, Broadcom, Accessed: May 7, 2026. [Online]. Available: <https://docs.spring.io/spring-boot/documentation.html>
- [33] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Chapter 5 defines the REST architectural style, Doctoral Dissertation, University of California, Irvine, Irvine, CA, USA, 2000, ch. 5: Representational State Transfer (REST). Accessed: May 7, 2026. [Online]. Available: https://roy.gbiv.com/pubs/dissertation/rest_arch_style.htm
- [34] The Apache Software Foundation. “Apache maven 3.9.6 release notes,” The Apache Software Foundation, Accessed: May 7, 2026. [Online]. Available: <https://maven.apache.org/docs/3.9.6/release-notes.html>
- [35] Tesseract OCR Project. “Tesseract ocr documentation (tessdoc).” Z. Podobny, Ed. Major Version 5.x, Tesseract OCR, Accessed: May 7, 2026. [Online]. Available: <https://tesseract-ocr.github.io/tessdoc/>
- [36] PostgreSQL Global Development Group. “Postgresql 15.0 documentation,” PostgreSQL Global Development Group, Accessed: May 7, 2026. [Online]. Available: <https://www.postgresql.org/docs/15/index.html>
- [37] MinIO, Inc. “Minio aistor documentation,” MinIO, Inc., Accessed: May 7, 2026. [Online]. Available: <https://docs.min.io/aistor/>

- [38] WHATWG. "Html living standard." I. Hickson, S. Pieters, A. van Kesteren, P. Jägenstedt, D. Denicola, and S. Montgomery, Eds., WHATWG, Accessed: May 7, 2026. [Online]. Available: <https://html.spec.whatwg.org/>
- [39] World Wide Web Consortium (W3C). "Cascading style sheets," W3C, Accessed: May 7, 2026. [Online]. Available: <https://www.w3.org/Style/CSS/>
- [40] Ecma International. "Standard ecma-262: EcmaScript 2026 language specification." I. Finlayson, M. Gibbard, J. Groff, C. Littlewood, and M. Shaver, Eds., Ecma International, Accessed: May 7, 2026. [Online]. Available: <https://ecma-international.org/publications-and-standards/standards/ecma-262/>
- [41] F5, Inc. "Nginx documentation." I. Sysoev, Ed., F5, Inc., Accessed: May 7, 2026. [Online]. Available: <https://nginx.org/en/docs/>
- [42] Docker, Inc. "Docker documentation," Docker, Inc., Accessed: May 7, 2026. [Online]. Available: <https://docs.docker.com/>
- [43] Compose Spec Community. "The compose specification," Commonhaus Foundation, Accessed: May 7, 2026. [Online]. Available: <https://www.compose-spec.io/>
- [44] Google. "Gemini api documentation," Google LLC, Accessed: May 7, 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs>
- [45] JetBrains s.r.o. "IntelliJ idea installation guide." Version 2026.1, JetBrains s.r.o., Accessed: May 7, 2026. [Online]. Available: <https://www.jetbrains.com/help/idea/installation-guide.html>
- [46] GitHub, Inc. "About git," GitHub, Inc., Accessed: May 7, 2026. [Online]. Available: <https://docs.github.com/en/get-started/using-git/about-git>
- [47] Docker, Inc. "Docker desktop documentation," Docker, Inc., Accessed: May 7, 2026. [Online]. Available: <https://docs.docker.com/desktop/>
- [48] OpenAI. "Chatgpt release notes," OpenAI, Accessed: May 7, 2026. [Online]. Available: <https://help.openai.com/en/articles/6825453-chatgpt-release-notes>
- [49] Google. "Gemini api changelog," Google LLC, Accessed: May 7, 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs/changelog>
- [50] Figma, Inc. "Explore figma make," Figma, Inc., Accessed: May 7, 2026. [Online]. Available: <https://help.figma.com/hc/en-us/articles/31304412302231-Explore-Figma-Make>
- [51] JetBrains s.r.o. "Junie: Your ai coding companion," JetBrains s.r.o., Accessed: May 7, 2026. [Online]. Available: <https://www.jetbrains.com/junie/>
- [52] Broadcom. "Scheduling and task execution." Version 7.0.7, Broadcom / VMware Tanzu, Accessed: May 7, 2026. [Online]. Available: <https://docs.spring.io/spring-framework/reference/integration/scheduling.html>

- [53] WHATWG community. "Fetch standard: Http-redirects and cors protocol." A. van Kesteren, Ed. Living Standard, Web Hypertext Application Technology Working Group, Accessed: May 7, 2026. [Online]. Available: <https://fetch.spec.whatwg.org/#http-cors-protocol>
- [54] Broadcom. "Spring data jpa reference documentation." Version 3.4.5, Broadcom / VMware Tanzu, Accessed: May 7, 2026. [Online]. Available: <https://docs.spring.io/spring-data/jpa/reference/index.html>
- [55] Broadcom. "Dependency injection: Dependency resolution process and collaborators." Version 6.2.x, Broadcom / VMware Tanzu, Accessed: May 7, 2026. [Online]. Available: <https://docs.spring.io/spring-framework/reference/core/beans/dependencies/factory-collaborators.html>
- [56] Jakarta EE Working Group. "Jakarta bean validation." M. Bekhta, Ed. Jakarta Validation 4.0 (Milestone 1), Eclipse Foundation, Accessed: May 7, 2026. [Online]. Available: <https://beanvalidation.org/>
- [57] Broadcom. "Spring boot actuator: Production-ready features." Version 4.0.5, Broadcom / VMware Tanzu, Accessed: May 7, 2026. [Online]. Available: <https://docs.spring.io/spring-boot/how-to/actuator.html>
- [58] WHATWG community. "Dom standard." A. van Kesteren, Ed. Living Standard, Web Hypertext Application Technology Working Group, Accessed: May 7, 2026. [Online]. Available: <https://dom.spec.whatwg.org/>
- [59] Lucide Contributors. "Lucide: Beautiful & consistent icons." E. Fennis, Ed. Version 1.14.0, Lucide Open Source Project, Accessed: May 7, 2026. [Online]. Available: <https://lucide.dev/>
- [60] WHATWG community. "Fetch standard." A. van Kesteren, Ed. Living Standard, Web Hypertext Application Technology Working Group, Accessed: May 7, 2026. [Online]. Available: <https://fetch.spec.whatwg.org/>
- [61] T. Bray. "The javascript object notation (json) data interchange format." RFC 8259, Standard 90, Internet Engineering Task Force, Accessed: May 7, 2026. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8259.html>
- [62] P. Leach, M. Mealling, and R. Salz. "A universally unique identifier (uuid) urn namespace." RFC 4122, Internet Engineering Task Force, Accessed: May 7, 2026. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc4122.html>
- [63] The Apache Software Foundation. "Apache pdfbox: An open source java tool for working with pdf documents." Version 3.0.3, The Apache Software Foundation, Accessed: May 7, 2026. [Online]. Available: <https://pdfbox.apache.org/>
- [64] Google. "Gemini api: Models and capabilities." Model variants: Flash, Pro, and Ultra, Google LLC, Accessed: May 7, 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs/models>

- [65] A. Wiggins. "The twelve-factor app: lii. config." Store config in the environment, Heroku, Accessed: May 7, 2026. [Online]. Available: <https://12factor.net/config>
- [66] S. Chacon and B. Straub, *Pro Git: Everything You Need to Know about Git* (The Expert's Voice), 2nd ed. New York, NY, USA: Apress, Dec. 2025, Version 2.1.449. Accessed: May 7, 2026. [Online]. Available: <https://git-scm.com/book/en/v2>

List of Figures

Figure 1 Alovre—upload interface.	11
Figure 2 Alovre—library interface.	11
Figure 3 Alovre—reading interface.	12
Figure 4 Alovre—UML component diagram.	13
Figure 5 Entity-relationship diagram of the Alovre system.	40
Figure 6 Upload interface mock-up generated with Figma Make.	41
Figure 7 Library interface mock-up generated with Figma Make.	42

List of Tables

Table 1	Feature comparison of prominent L2 reading platforms.	3
Table 2	Overview of use cases.	8
Table 3	Functional requirements of the AI-assisted reading system.	9
Table 4	Non-functional requirements of the AI-assisted reading system.	9
Table 5	Functional test cases.	19
Table 6	Non-functional test cases.	20
Table 7	Requirements traceability matrix.	20
Table 9	Heuristic evaluation results: document upload.	45
Table 10	Heuristic evaluation results: open document.	46
Table 11	Heuristic evaluation results: translation and explanation.	47
Table 12	Heuristic evaluation results: image generation.	49
Table 13	Heuristic evaluation results: TTS pronunciation.	50

Documentation table of AI-based tools

AI-Based Tool	Intended Use	Prompt, Source, Page, Paragraph. . .
ChatGPT 4.0 / Gemini 3 Flash	Academic tone and stylistic refinement	"Rewrite this paragraph in an academic tone without modifying the information in it: [Paragraph]", all cases of referencing a source
ChatGPT 4.0 / Gemini 3 Flash	Code generation (frontend/backend)	"Please write the frontend/backend code for this functionality: [Function Name]", Chapter 3: Solution
ChatGPT 4.0 / Gemini 3 Flash	Code debugging and error resolution	"Please fix this error: [Error Log]", Used during the development of the prototype
Figma Make	Automated UI prototyping	"Please create a frontend prototype with these functionalities: [List]", Appendix C: User Interface Design Artefacts
ChatGPT 4.0 / Gemini 3 Flash	Content review and structural feedback	"What do you think of this section of the thesis? Are there any mistakes or room for improvement?", Applied to various sections during final editing

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
DSR	Design Science Research
L1	First Language
L2	Second Language
LLM	Large Language Model
OCR	Optical Character Recognition
REST	Representational State Transfer
RTM	Requirements Traceability Matrix
TTS	Text-to-Speech
UML	Unified Modelling Language

A Project Setup and Execution

This appendix describes how to set up and run Alovre. To ensure all dependencies (such as Tesseract OCR) are correctly configured, the system is designed to run exclusively within a containerised environment.

The complete source code is available via GitHub: <https://github.com/if23b196/alovre.git>

A.1 Running the Application (Docker)

The system utilises Docker to orchestrate the Spring Boot backend, NGINX frontend, PostgreSQL database, and MinIO storage into a single functional stack.

Prerequisites

- **Docker Desktop:** Installed and running.
- **Gemini API Key:** A valid key obtained from Google AI Studio.
- **Port Availability:** Ensure ports 8080, 8081, 9000, and 9001 are not occupied by other local services.

Setup Steps

1. Obtain the Source Code:

- Either clone the repository using Git: `git clone https://github.com/if23b196/alovre.git`
- Or download the repository as a ZIP file from GitHub and extract it to a local folder.

2. Configure Environment: Create a file named `.env` in the project root directory and add your API key:

```
GEMINI_API_KEY=your_actual_api_key_here
```

3. Launch the Stack: Open a terminal in the project root and execute:

```
docker compose up --build
```

Note: The initial build may take several minutes as it installs the Java runtime and Tesseract OCR dependencies.

Accessing the Application

Once the terminal indicates that all containers are healthy, the services are available at:

- **Frontend (Alovre Reader):** `http://localhost:8081`
- **Backend API:** `http://localhost:8080`
- **MinIO Storage Console:** `http://localhost:9001`

Security and Credentials

As this is a research prototype, infrastructure credentials (PostgreSQL and MinIO) are pre-configured within the `application.properties` and `docker-compose.yml` files for ease of use. No additional database setup or manual credential configuration is required by the user.

A.2 Modifying the Project

To modify Alovre, the source code can be edited using an IDE. However, the system must always be executed through Docker; running the backend directly in IntelliJ will lead to errors due to missing system dependencies and environment configurations.

Prerequisites

- **Java Development Kit (JDK) 21**
- **IntelliJ IDEA:** Recommended for working with the backend code.

Development Workflow

1. **Open Project:** In IntelliJ IDEA, open the `backend` folder and select the `pom.xml` file to load it as a Maven project.
2. **Make Changes:** Edit the Java files in the backend or the HTML/CSS/JS files in the frontend folder as needed.
3. **Apply and Run:** Because the application relies on the containerised environment, every change to the backend code requires a rebuild of the Docker containers to take effect. Use the following commands in the project root:

```
docker compose down
docker compose up --build
```

4. **Frontend Updates:** Changes to the frontend files are usually applied instantly. Simply refresh the browser at `http://localhost:8081` while the Docker containers are running.

System Reset

To completely reset the system and delete all stored documents, AI-generated images, and database entries, run:

```
docker compose down -v
```

This removes the containers along with their persistent data volumes, allowing for a clean restart.

B Data Model and Persistence

This appendix describes the relational database structure of Alovre, managed via Spring Data JPA and PostgreSQL. The system uses a centralised model where all AI-generated resources are linked to a primary document.

B.1 Entity Overview

The persistence layer consists of four main entities:

- **Content:** Stores document metadata (title, language, type), the extracted `textContent`, and references to the original file in MinIO.
- **Annotation:** Stores AI-generated translations and explanations linked to specific user selections.
- **Image:** Maintains references to contextual images stored in MinIO.
- **Audio:** Manages metadata for TTS outputs, including the voice used and the file reference.

B.2 Relationships and Schema

The model implements a one-to-many ($1 : N$) relationship between `Content` and the supporting multimodal entities. As shown in the entity-relationship diagram, deleting a `Content` entry triggers a cascade that automatically removes all associated annotations, images, and audio records to ensure data integrity.

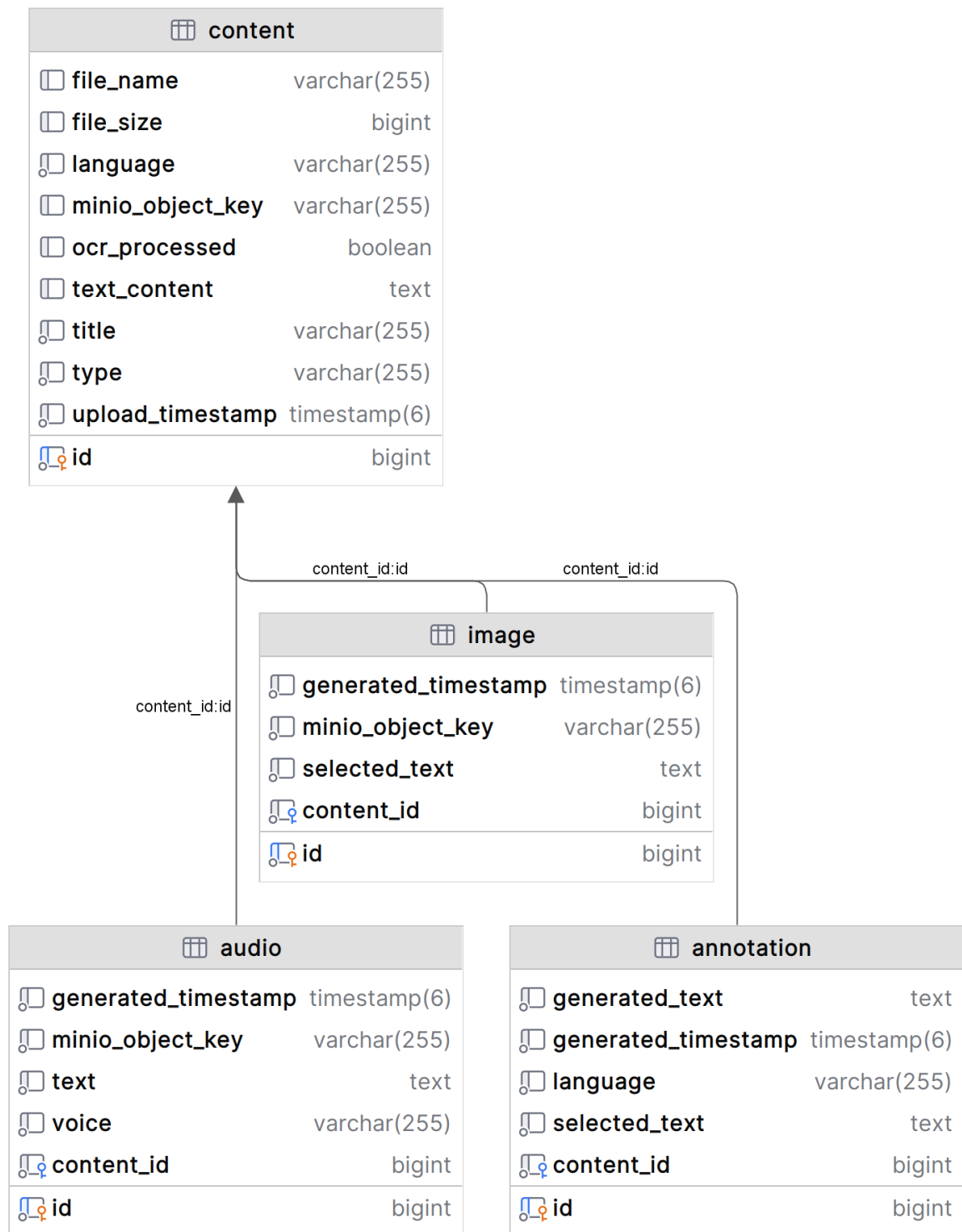


Figure 5: Entity-relationship diagram of the Alovre system.

C User Interface Design Artefacts

This appendix provides supplementary material related to the design and prototyping of the user interface described in Section 3.5.3. The interface was developed with a focus on clarity, readability, and intuitive interaction.

During the early design phase, AI-supported prototyping tools (Figma Make) were used to generate initial interface concepts. Through iterative refinement of various prompts, the layout, visual hierarchy, and interaction patterns were progressively improved to serve as a conceptual foundation for the final implementation. Core elements established during this phase were subsequently transferred and adapted into the working frontend.

The provided figures illustrate how early prototype designs informed the functional user interface, documenting the transition from the initial concept to the final implementation.

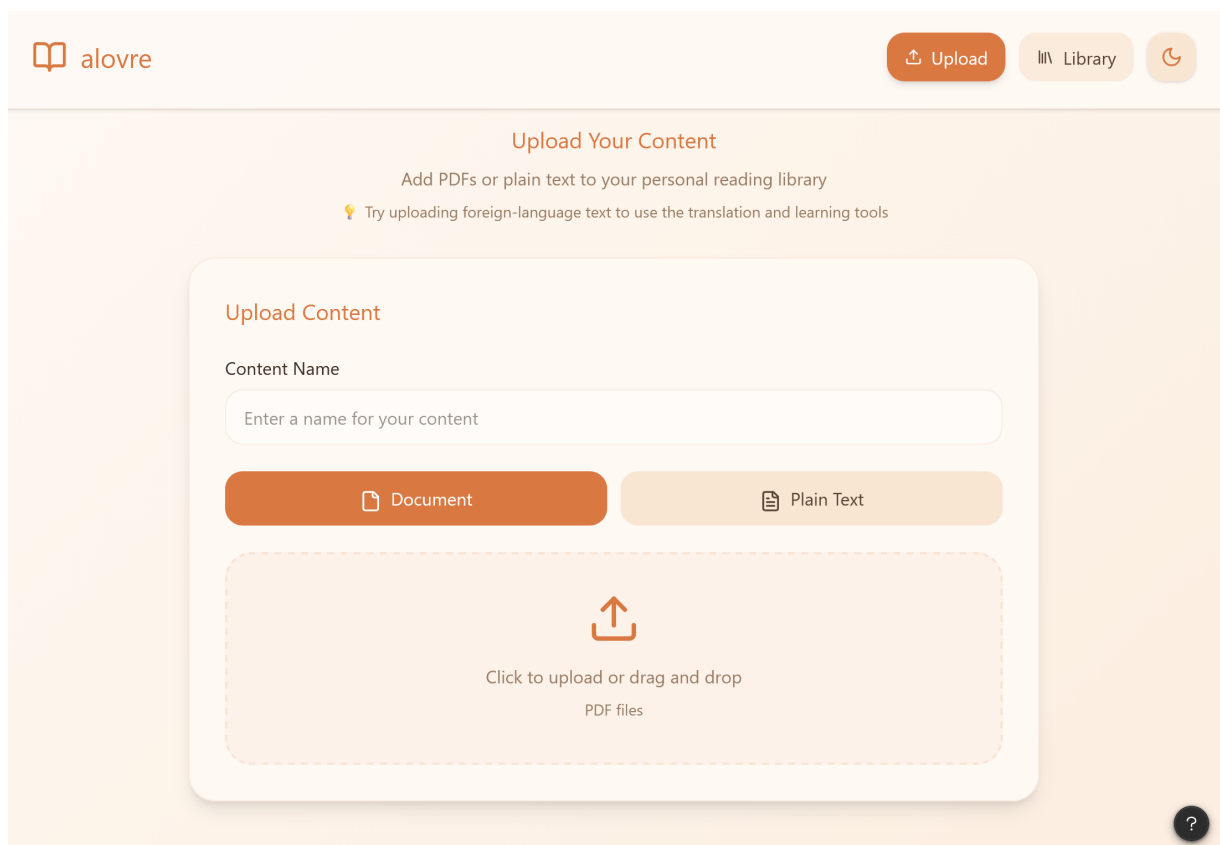


Figure 6: Upload interface mock-up generated with Figma Make.

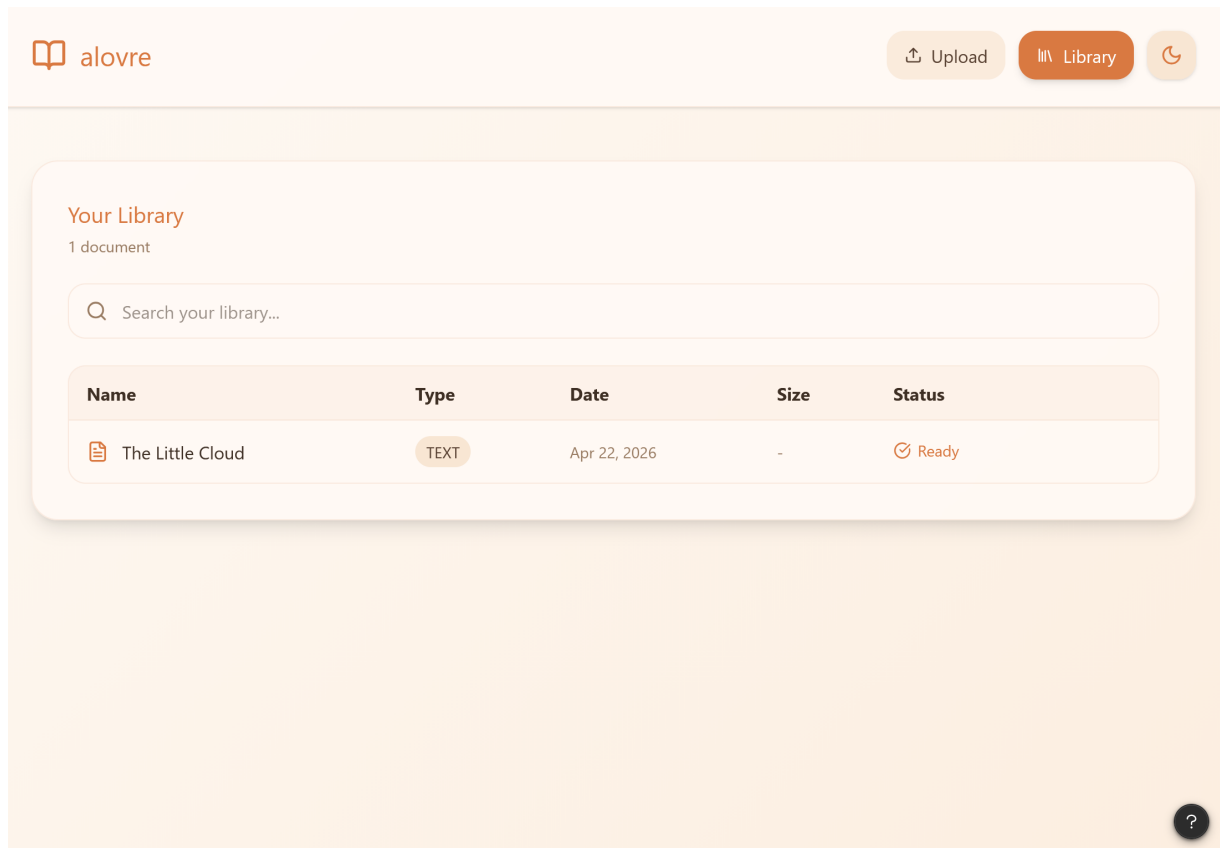


Figure 7: Library interface mock-up generated with Figma Make.

D AI Prompt Design

This appendix summarises how prompts are used to guide the integrated AI services. Rather than relying on unrestricted model behaviour, the system provides structured instructions that define the expected task, contextual information, and desired output characteristics.

D.1 Translation and Explanation Prompt

The translation feature uses prompts that instruct the language model to *“translate the word using the context below”* and to *“explain the word very briefly in one sentence”*. The selected word and its surrounding sentence context are included to help the model determine the intended meaning of ambiguous terms.

The prompt dynamically incorporates both the language of the uploaded content and the user’s selected target language. The translation is generated in the user’s selected language (L1), while the explanation remains in the language of the original document (L2).

Furthermore, the prompt explicitly requests *“only valid JSON”* output in order to ensure that responses can be processed reliably by the backend system.

D.2 Image Generation Prompt

The image generation feature uses prompts specifically designed for educational vocabulary support. The model is instructed to *“create a clear, simple, and visually unambiguous image”* representing the selected word.

To improve semantic accuracy, the prompt emphasises that the model should *“use the context to determine the correct meaning of the word”* and *“choose only the meaning that fits the context”* when multiple interpretations exist.

Additional instructions such as *“focus on one main subject”*, *“avoid text inside the image”*, and *“use a clean, realistic, or simple illustration style”* are intended to improve visual clarity and interpretability, while reducing unnecessary visual complexity for language learners.

D.3 Text-To-Speech Prompt

The TTS functionality uses a minimal prompt focused on pronunciation clarity. The instruction *“Pronounce clearly”* directs the model to generate understandable spoken output for the

selected word without adding additional explanations or conversational content.

E Heuristic Evaluation Details

This appendix provides a detailed breakdown of the heuristic evaluation results for all evaluated system modules and interaction workflows.

Table 9 presents the detailed heuristic evaluation of the document upload use case.

Table 9: Heuristic evaluation results: document upload.

ID	Heuristic	Problem Description	Severity	Recommendation
H1	Visibility of System Status	No detailed progress indicator (only spinner) during large file uploads.	1	Add progress bar (0–100%) to improve upload transparency.
H2	Match Between System and Real World	Terminology such as "Document", "Library", and "Plain Text" is clear and user-friendly.	0	No change required.
H3	User Control and Freedom	Users can remove selected files before upload using the cancel option.	0	No change required.
H4	Consistency and Standards	The upload button is disabled until required fields are completed, but the reason is not explicitly communicated.	1	Add clarification for disabled state (e.g., missing input indicator).
H5	Error Prevention	File type restrictions and input validation prevent invalid uploads.	0	No change required.
H6	Recognition Rather Than Recall	Selected file name is clearly displayed to the user.	0	No change required.
H7	Flexibility and Efficiency of Use	Supports both drag-and-drop and manual file selection.	0	No change required.
Continued on next page...				

Continuation of Table 9				
ID	Heuristic	Problem Description	Severity	Recommendation
H8	Aesthetic and Minimalist Design	The interface is clean and focuses only on required input fields.	0	No change required.
H9	Help Users Recognise and Recover from Errors	Error messages such as "Upload failed" are too generic.	3	Provide specific error feedback (e.g., network or file size issues).
H10	Help and Documentation	Basic guidance exists, but contextual help is limited.	1	Add tooltips or inline hints for upload constraints.

Table 10 summarises the identified usability issues for the open document workflow. No major usability violations were detected.

Table 10: Heuristic evaluation results: open document.

ID	Heuristic	Problem Description	Severity	Recommendation
H1	Visibility of System Status	A spinner and reduced opacity indicate processing state, clearly signaling that content is loading.	0	No action required.
H2	Match Between System and Real World	Technical labels (e.g., TEXT, PDF, TXT) use database-style formatting instead of user-friendly language.	1	Replace with natural language labels (e.g., "Plain text").
H3	User Control and Freedom	Navigation and back actions allow users to exit the document view easily.	0	No action required.
H4	Consistency and Standards	Entire list rows function as clickable elements, consistent with common web patterns.	0	No action required.
H5	Error Prevention	Editing and deletion actions are disabled during processing to prevent invalid interactions.	0	No action required.
Continued on next page...				

Continuation of Table 10				
ID	Heuristic	Problem Description	Severity	Recommendation
H6	Recognition Rather Than Recall	Document title remains visible in the reader interface for continuous context awareness.	0	No action required.
H7	Flexibility and Efficiency of Use	UI updates rely on automatic polling, but no manual refresh option is available.	1	Add manual refresh control for improved responsiveness.
H8	Aesthetic and Minimalist Design	Action buttons are conditionally displayed, keeping the interface uncluttered.	0	No action required.
H9	Help Users Recognise and Recover from Errors	Missing or deleted documents accessed via direct links are not explicitly handled in the UI.	1	Add a "Document not found" page with navigation back to library.
H10	Help and Documentation	Empty state provides clear guidance for first-time users.	0	No action required.

Table 11 presents the detailed heuristic evaluation of the translation and explanation use case.

Table 11: Heuristic evaluation results: translation and explanation.

ID	Heuristic	Problem Description	Severity	Recommendation
H1	Visibility of System Status	A loading toast with spinner is shown during AI processing, providing immediate feedback.	0	No action required.
H2	Match Between System and Real World	The selected word remains visibly displayed within the translation panel, maintaining a clear association between the user selection and the generated output.	0	No action required.
Continued on next page...				

Continuation of Table 11				
ID	Heuristic	Problem Description	Severity	Recommendation
H3	User Control and Freedom	Users can navigate generated translations and explanation via pagination, but cannot remove individual entries.	2	Add option to delete or clear translations from history.
H4	Consistency and Standards	Interaction patterns are consistent across translation, image, and audio tools.	0	No action required.
H5	Error Prevention	AI actions are disabled until a valid word selection is made.	0	No action required.
H6	Recognition Rather Than Recall	The selected word is highlighted in the text and shown in the tool panel.	0	No action required.
H7	Flexibility and Efficiency of Use	The distance between text selection and action buttons increases interaction effort.	2	Introduce shortcuts (e.g., double-click or keyboard shortcut for translation).
H8	Aesthetic and Minimalist Design	Result area adapts dynamically and maintains a clean visual layout.	0	No action required.
H9	Help Users Recognise and Recover from Errors	Error feedback is shown via toast notifications, but no in-component retry option exists.	1	Add a retry button inside the result panel for failed requests.
H10	Help and Documentation	Initial UI state provides clear instructions for using AI tools.	0	No action required.

Table 12 presents the detailed heuristic evaluation of the image generation use case.

Table 12: Heuristic evaluation results: image generation.

ID	Heuristic	Problem Description	Severity	Recommendation
H1	Visibility of System Status	Third-party API latency causes unclear processing status without persistent in-component feedback.	1	Introduce an indefinite loading state (e.g., skeleton screen) within the image container.
H2	Match Between System and Real World	The system uses a word-based badge that clearly labels generated images with their source term.	0	No action required.
H3	User Control and Freedom	Users can navigate generated images via pagination, but cannot remove individual entries.	2	Add option to delete or clear individual images from history.
H4	Consistency and Standards	The interaction pattern is consistent across translation, image, and audio tools.	0	No action required.
H5	Error Prevention	Image generation is only enabled after valid word selection, including contextual input enrichment.	0	No action required.
H6	Recognition Rather Than Recall	Generated images are labeled with their corresponding word, maintaining context visibility.	0	No action required.
H7	Flexibility and Efficiency of Use	Each image must be generated individually, increasing effort during multi-word usage.	2	Enable batch generation or a gallery view for multiple images.
H8	Aesthetic and Minimalist Design	Inconsistent image aspect ratios lead to visual misalignment and letterboxing effects.	1	Standardise image container scaling to maintain visual consistency.
H9	Help Users Recognise and Recover from Errors	Failed image loads are only indicated via toast messages, without in-component recovery options.	1	Add broken-image placeholders with retry functionality.
Continued on next page...				

Continuation of Table 12				
ID	Heuristic	Problem Description	Severity	Recommendation
H10	Help and Documentation	Instructional text supports user understanding of the feature.	0	No action required.

The complete evaluation of the TTS pronunciation use case is presented in Table 13.

Table 13: Heuristic evaluation results: TTS pronunciation.

ID	Heuristic	Problem Description	Severity	Recommendation
H1	Visibility of System Status	The system shows a global loading spinner during audio synthesis, while the tool card remains static until completion.	1	Move loading feedback into the audio result area with an indefinite or pulsing animation.
H2	Match Between System and Real World	The play button clearly indicates the target word (e.g., Play "Example"), improving semantic clarity.	0	No action needed.
H3	User Control and Freedom	Users can revisit previous pronunciations via pagination, but cannot remove individual audio entries.	2	Add a delete or clear option for individual audio history items.
H4	Consistency and Standards	The interaction pattern is fully consistent with Translation and Image tools.	0	No action needed.
H5	Error Prevention	The pronunciation trigger is disabled until a valid word selection is made.	0	No action needed.
H6	Recognition Rather Than Recall	The source word is explicitly shown on the generated audio element, avoiding ambiguity.	0	No action needed.
H7	Flexibility and Efficiency of Use	The two-step process (generate then play) introduces interaction overhead for frequent use.	2	Introduce an auto-play option after synthesis completion.

ID	Heuristic	Problem Description	Severity	Recommendation
H8	Aesthetic and Minimalist Design	The audio output adapts to a minimal floating layout, reducing visual clutter.	0	No action needed.
H9	Help Users Recognise and Recover from Errors	Failures are only communicated via toast notifications without a retry option in the audio card.	1	Add an inline retry button within the audio component.
H10	Help and Documentation	The placeholder text and volume icon clearly communicate the function of the tool.	0	No action needed.